

BURHANI COLLEGE

— ESTD. 1970 —

Burhani College,10 Nesbit Road, Mazgaon, Mumbai 400010.

B. Sc. (Computer Science)

Course Code:	U	S	C	S	P	5		

Course Name: _____ Practical

Name of the Student: (Mr./Miss)_____

Roll No: _____ Seat Number: _____ Class: T. Y. B. Sc. CS

Submission Date: _____ Teachers Sign: _____

B. Sc. (Computer Science)

T. Y. B. Sc. CS

CERTIFICATE

This is to certify that the practical work , USCSP5_____ "_____

Practical", is bonafied work of (Mr./Miss) _____ bearing

University Exam Seat Number: _____, College Roll Number:

_____, submitted in partial fulfillment of the requirements for the award of degree of

BACHELOR OF SCIENCE in (COMPUTER SCIENCE) from University of Mumbai.

_____ _____

Subject Teacher Coordinator Sign

_____ _____ _____ Principal Sign External Signature

College Seal

INDEX

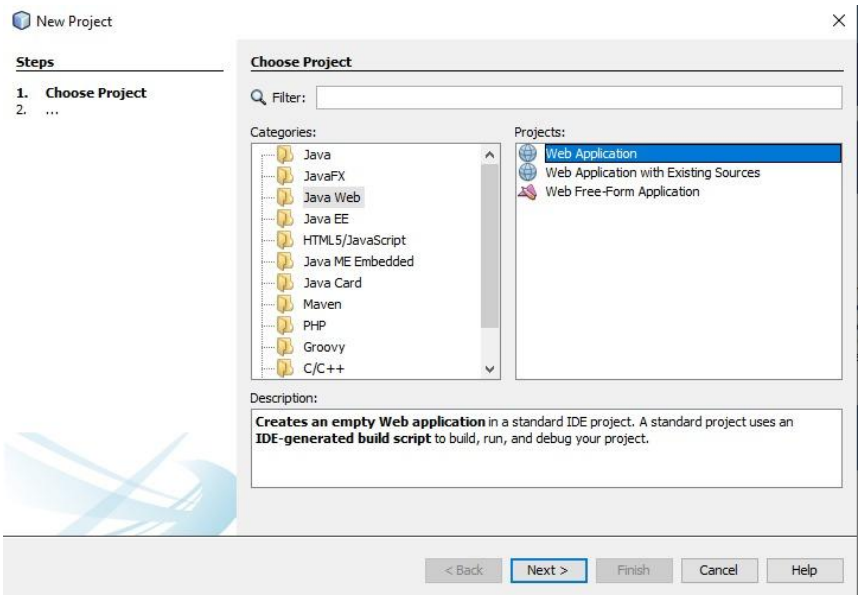
Sr.no	Date	Title	Sign
1.	08/12/25	Define a simple services like converting Indian Rupees to dollar and call it from different platforms like java and .Net	
2.	15/12/25	Create a simple SOAP services	
3.	05/01/26	To create a Simple RestFul web service without using a database that perform string operations(uppercase and lowercase)	
4.	12/01/26	Develop application to consume Google's search / Google's Map RESTful web service.	
5.	19/01/26	Develop application to download image/video from server or upload image/video	
6.	02/02/26	Installation and Configuration of virtualization using KVM	
7.	23/02/26	Develop and deploy a Servlet-based web application on Apache Tomcat	
8.	02/03/26	Develop a RESTful web service using java that monitors and provides system resource usage, especially memory and CPU usage, and returns this information in JSON format via HTTP GET requests	

PRACTICAL 1

Aim: Define a simple services like converting Indian Rupees to dollar and call it from different platforms like java and .Net

Step 1: In Netbeans, Go to File -> New Project -> Java Web, select Web Application -> Next

Project Name: Practical1 -> Next -> Finish



Step 2: Right Click on Practical1 -> New -> Web Services

Web Service name: currency_converter

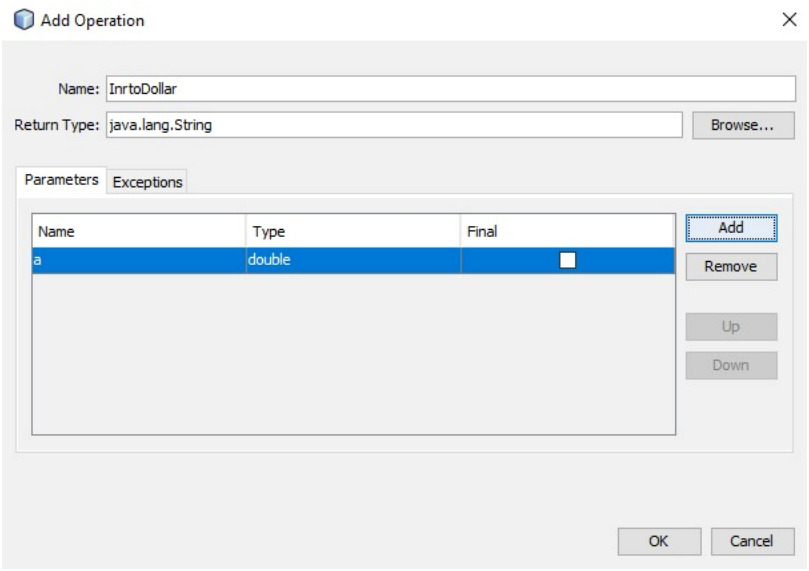
Package name: server1

Step 3: In currency_converter file, empty contents of public class. Right click there ->

Insert Code -> Add Web Service operation

Name: InrtoDollar

Add this and click OK



Step 4: Edit the code. Remove null from return null.

Add:

return “The Indian rupees” +a+ “Dollar is:” + (a/89.83)

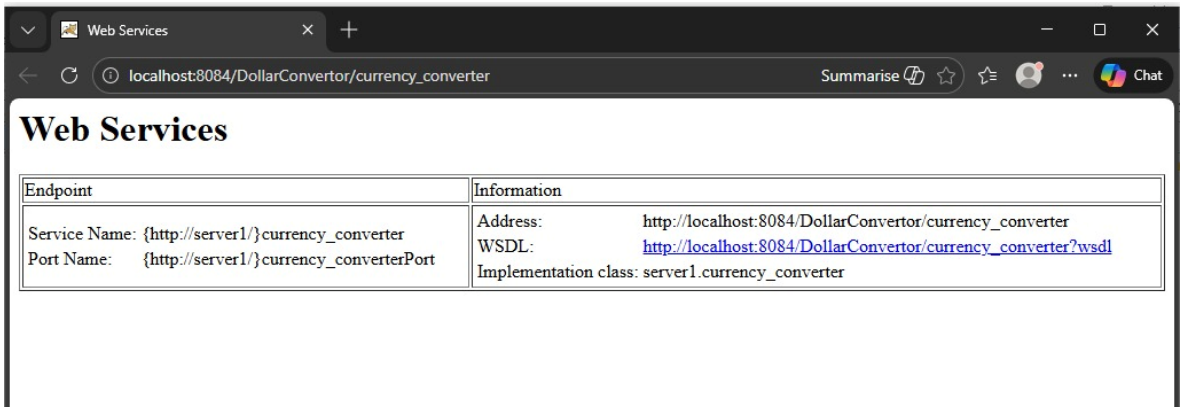
```
@WebService(serviceName = "currency_converter")
public class currency_converter {

    /**
     * Web service operation
     */
    @WebMethod(operationName = "InrtoDollar")
    public String InrtoDollar(@WebParam(name = "a") double a) {
        //TODO write your implementation code here:
        return "The indian rupees"+a+"in dollars:"+(a/89.83);
    }

}
```

Right click on Practical1 -> Deploy

Step 5: Go to Web Services folder -> currency_converter -> right click -> Test Web Services



Step 6: Right click on Web pages folder -> New -> JSP

File name: input

File name: output

Similarly create file for output

Step 7: Go to input.jsp file. Remove content of body.

Write the following code,

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
    </head>
    <body>
        <form action ="output.jsp">
            <pre>
                Enter your Indian rupees:
                <input type="text" name="tl">
                <input type="Submit">
                <input type="Reset"
            </pre>
        </form>
    </body>
</html>
```

Step 8: Right click on Practical1 -> New -> Other -> Web Services -> Web Services client -> Next

Browse -> Expand Practical1 and select currency_converter

Package name: client

Step 9: Web Service References folder -> InrtoDollar

Empty contents of body then drag InrtoDollar in output.jsp file.

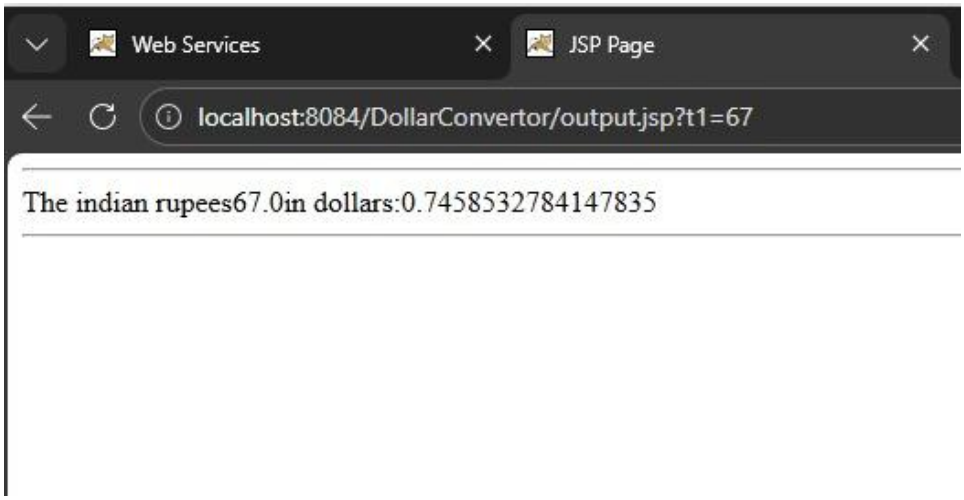
Change:

```
<body>
        <%-- start web service invocation --%><hr/>
<%
try {
    client.CurrencyConverter_Service service = new client.CurrencyConverter_Service();
    client.CurrencyConverter port = service.getCurrencyConverterPort();
     // TODO initialize WS operation arguments here
    double a = Double.parseDouble(request.getParameter("tl"));
    // TODO process result here
    java.lang.String result = port.inrtoDollar(a);
    out.println(result);
} catch (Exception ex) {
    // TODO handle custom exceptions here
}
%>
<%-- end web service invocation --%><hr/>

</body>
```

Deploy Practical1 then run input.jsp file

Output:



PRACTICAL 2

Aim: Create a simple SOAP services

Step 1: Open Netbeans. Go to File -> New Project -> Java Web, select Web Application -> Next

Project Name: Practical2 -> Next -> Finish

Step 2: Right Click on Practical2 -> New -> Web Services

Web Service name: soap_services

Package name: server1

Step 3: In soap_services file, empty contents of public class. Right click there ->

Insert Code -> Add Web Service operation

Name: tempConv

Add this and click OK

Step 4: Edit the code. Remove null from return null.

Add:

return “The Fahrenheit temperature” +a+ “in Celsius is:” + ((a-32)*5/9);

```
15      */
16      @WebService(serviceName = "pract2")
17      public class pract2 {
18
19          /**
20           * Web service operation
21           */
22          @WebMethod(operationName = "soapServe")
23          public String soapServe(@WebParam(name = "a") double a) {
24              //TODO write your implementation code here:
25              return "The Fahrenheit Temperature " + a + " in celsius is : " + ((a-32)*5/9);
26          }
27
28
29      }
```

Right click on Practical1 -> Deploy

Expand Web Services folder ->soap -> Test Web Service

Step 5: Go to Web Services folder -> tempConv -> right click -> Test Web Services

Web Services

Endpoint	Information
Service Name: {http://server1/}pract2 Port Name: {http://server1/}pract2Port	Address: http://localhost:8084/soap/pract2 WSDL: http://localhost:8084/soap/pract2?wsdl Implementation class: server1.pract2

Step 6: Right click on Web pages folder -> New -> JSP

File name: input

File name: output

Similarly create file for output

Step 7: Go to input.jsp file. Remove content of body.

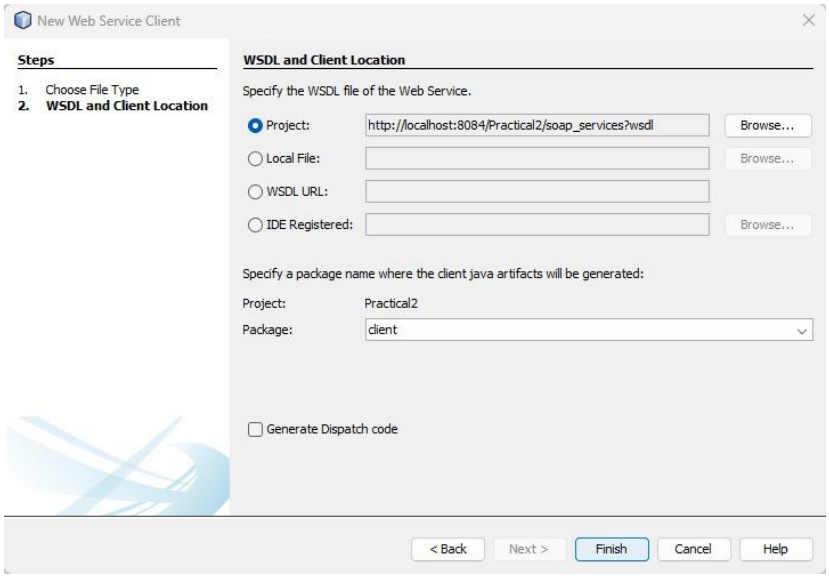
Write the following code,

```
<body>
    <form action =" output.jsp">
        <pre>
            Enter the temperature in Fahrenheit : <input type='text' name='t1' required>
            Enter the temperature in Celsius : <input type='text' name='t2' required>
            <input type='submit'>
            <input type='reset'>
        </pre>
    </form>
</body>
```

Step 8: Right click on Practical2 -> New -> Other -> Web Services -> Web Services client -> Next

Browse -> Expand Practical2 and select tempConv

Package name: client



Step 9: Expand Web Service References folder -> Select tempConv

Empty contents of body then drag tempConv file in output.jsp file.

Change:

```
<body>
        <%-- start web service invocation --%><hr/>
<%
try {
    client.CurrencyConverter_Service service = new client.CurrencyConverter_Service();
    client.CurrencyConverter port = service.getCurrencyConverterPort();
     // TODO initialize WS operation arguments here
    double a = Double.parseDouble(request.getParameter("tl"));
    // TODO process result here
    java.lang.String result = port.inrtoDollar(a);
    out.println(result);
} catch (Exception ex) {
    // TODO handle custom exceptions here
}
%>
<%-- end web service invocation --%><hr/>

</body>
```

Deploy Practical2 then run input.jsp file

Enter the temperature in Fahrenheit : 67

Enter the temperature in Celsius : 98

Submit

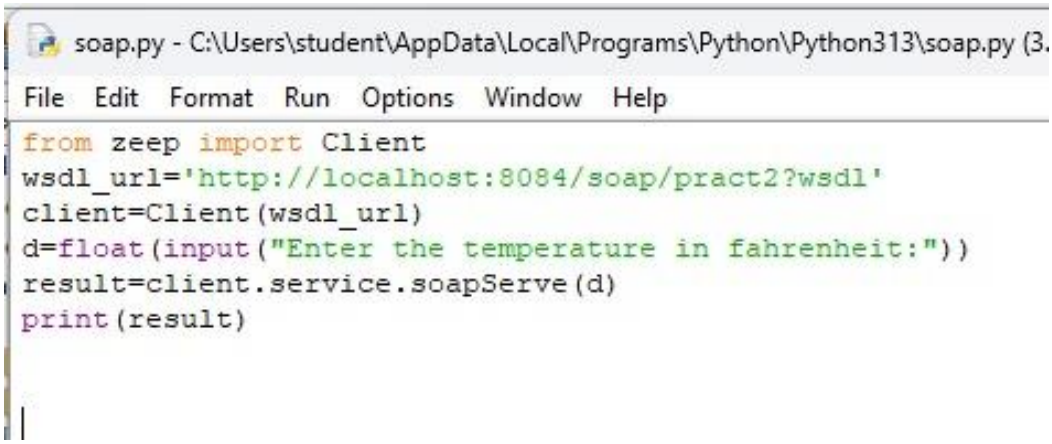
Reset

Output:

The Fahrenheit Temperature 67.0 in celsius is : 19.444444444444443

In step 8, copy the url after selecting tempConv file

Step 10: Run this code in Python IDLE

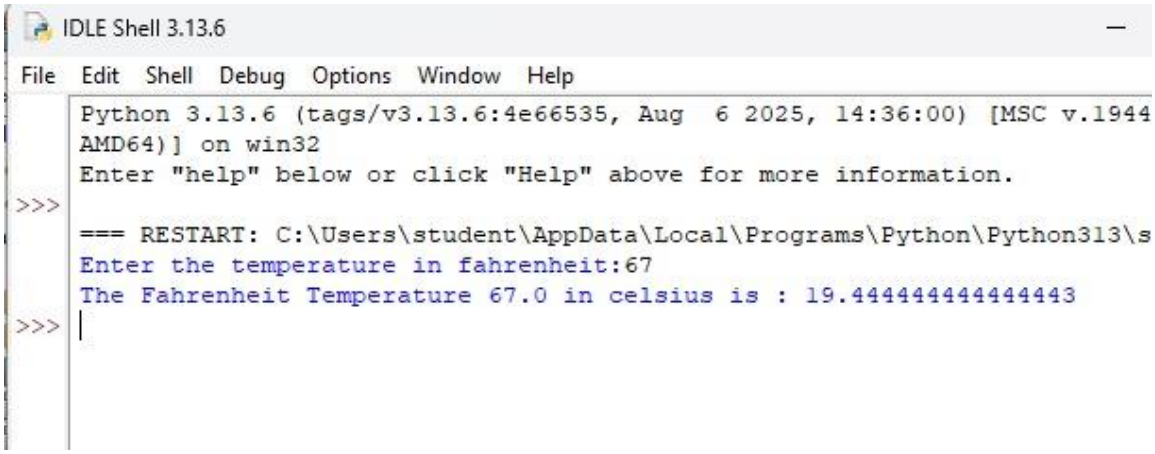


Replace wsdl_url with your url

Replace soapServe with your actual file name under:

Web Service References/Practical2/Practical2/Practical2Port/soapServe

Output:



PRACTICAL 3

Aim: To create a Simple RestFul web service without using a database that perform string operations(uppercase and lowercase)

Step 1 -> Create New project -> Java Web, select Web Application

Project name: RestFulServices

Step 2: Right click on RestFulServices folder -> New -> RestFul Web Services from pattern

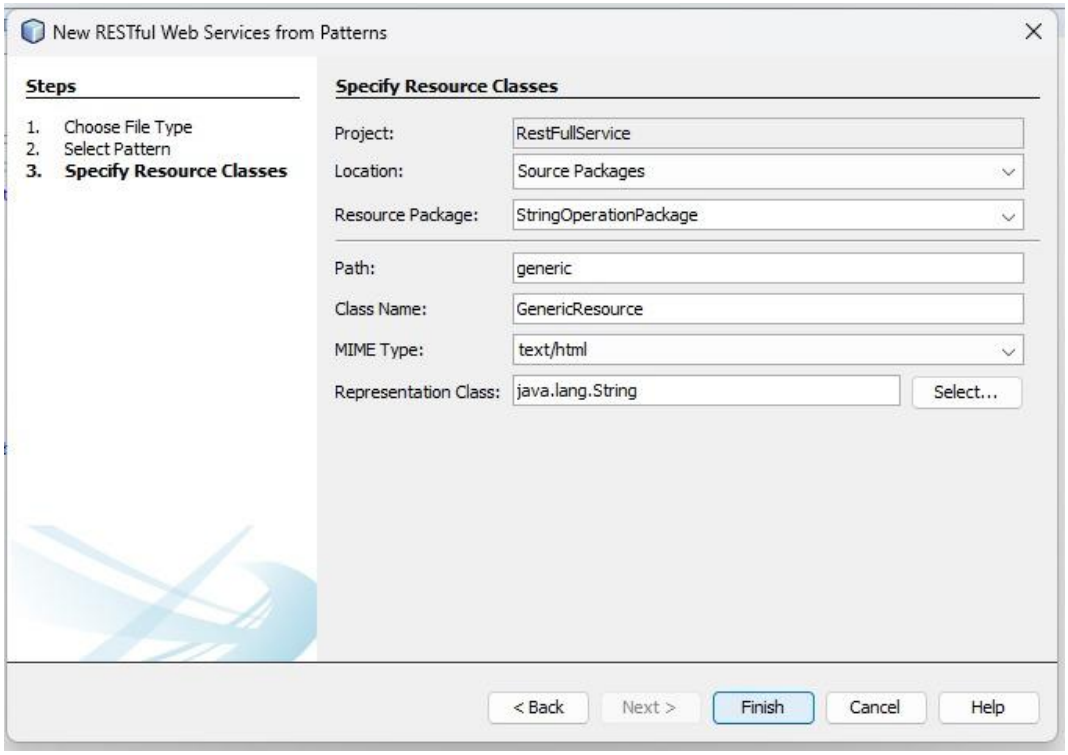
-> Simple Root Resource -> Next

Resource package: StringOperationPackage

Path: generic

MIME type: text/html

Then click Finish



Step 3: In the file, after @PUT

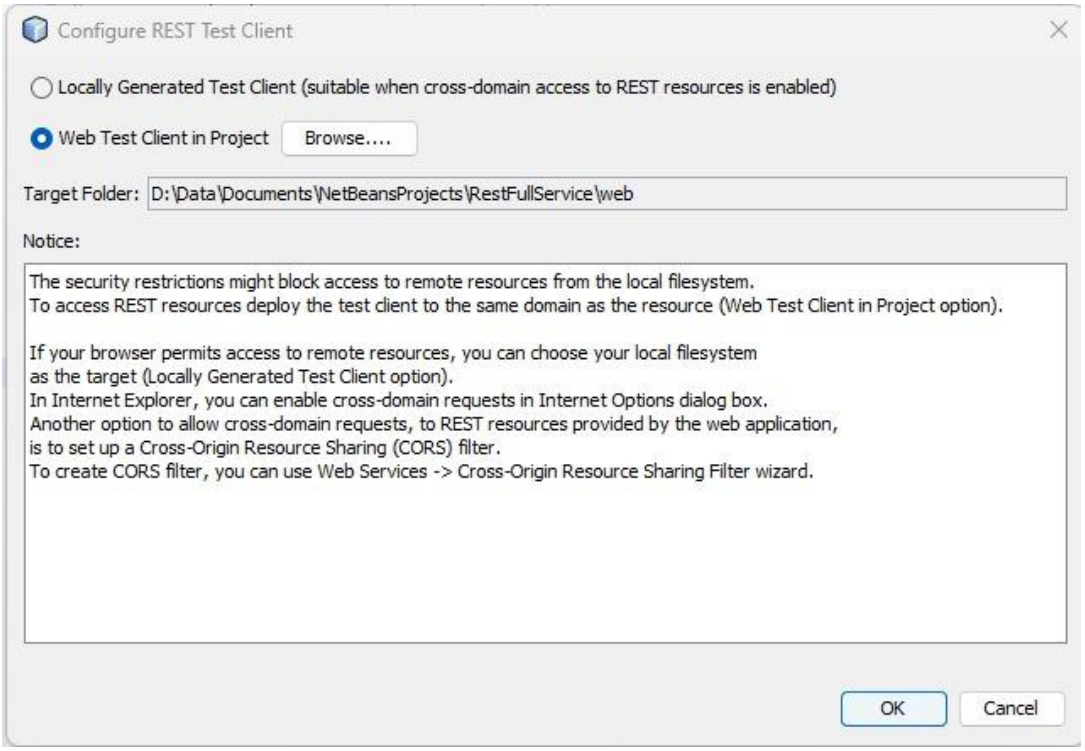
Write the following code,

```
    @PUT
    @Consumes(MediaType.TEXT_HTML)
    @Path("/Uppercase")
    public String toUppercaseMethod(String str){
        return str.toUpperCase();
    }

    @PUT
    @Consumes(MediaType.TEXT_HTML)
    @Path("/Lowercase")
    public String toLowercaseMethod(String str){
        return str.toLowerCase();
    }
}
```

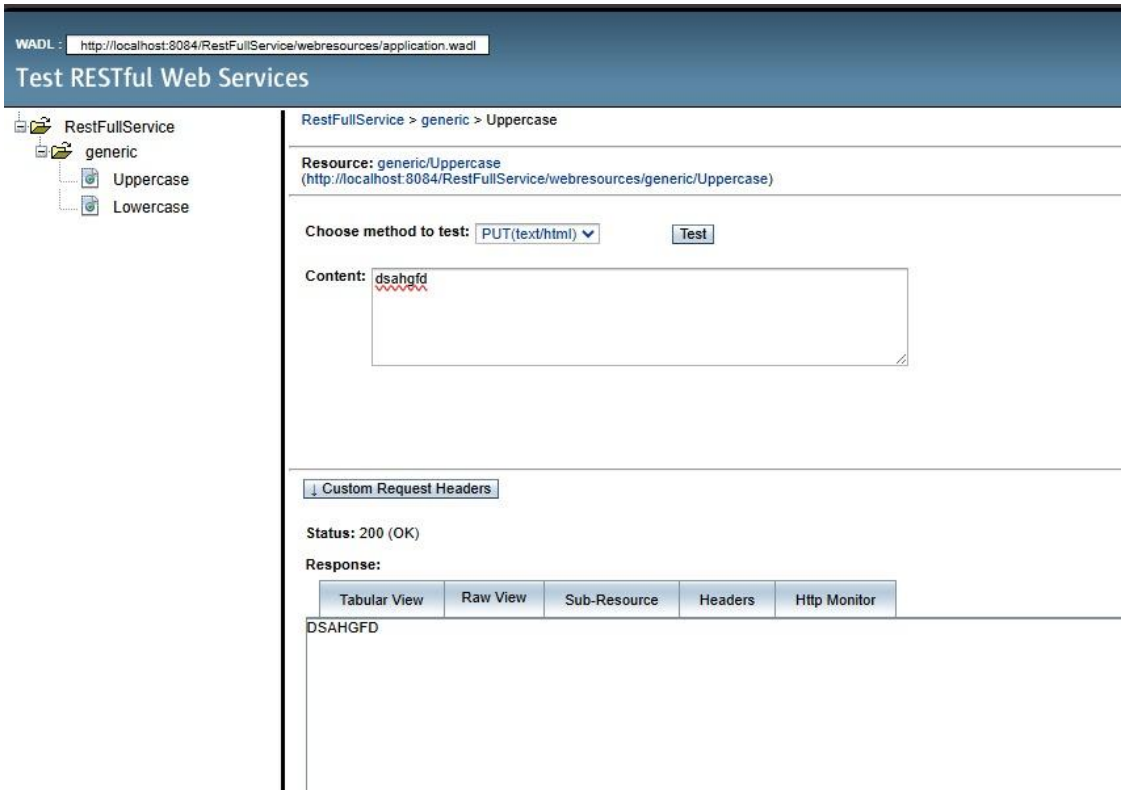
Right Click on RestFulServices -> Clean, Deploy

Then right Click again -> Test Restful Web Services -> OK



Write Something in the Text boxes to Test

Output:

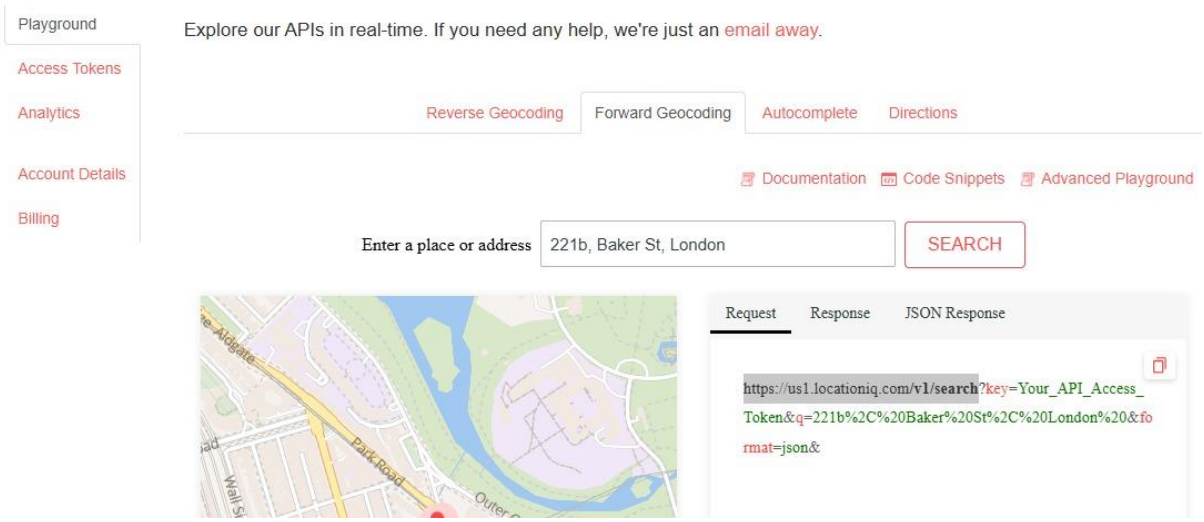

WADL : http://localhost:8084/RestFullService/webresources/application.wadl
Test RESTful Web Services
RestFullService
generic
Uppercase
Lowercase
RestFullService > generic > Uppercase
Resource: generic/Uppercase
(http://localhost:8084/RestFullService/webresources/generic/Uppercase)
Choose method to test: PUT(text/html)
Test
Content: dsahgfd
↓ Custom Request Headers
Status: 200 (OK)
Response:
Tabular View
Raw View
Sub-Resource
Headers
Http Monitor
DSAHGFD

PRACTICAL 4

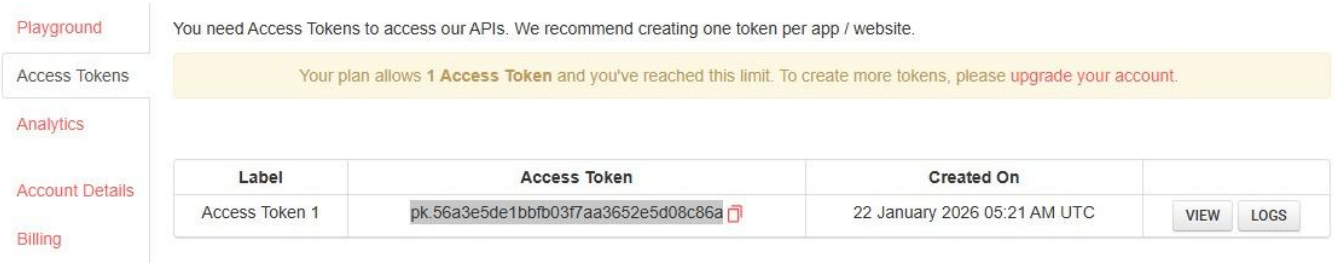
Aim: Develop application to consume Google's search / Google's Map RESTful web service.

Step 1: Go to locationiq.com website. Sign up with your email id

Step 2: Go to Playground tab -> Forward Geocoding -> Request -> Copy and replace it with base_url in the code



Step 3: Go to Access Tokens tab -> Copy the token and replace it with api_key in the code



Step 4: In Python IDLE, run this code

```
import requests

def get_geolocation(api_key, search_string):
    base_url = "https://us1.locationiq.com/v1/search"

    params = {
        'key': api_key,
        'q': search_string,
        'format': 'json'
    }

    response = requests.get(base_url, params=params)
    data = response.json()

    if response.status_code == 200 and data:
        result = {
            'place_id': data[0].get('place_id', ''),
            'lat': data[0].get('lat', ''),
            'lon': data[0].get('lon', ''),
            'display_name': data[0].get('display_name', '')
        }
        return result
    else:
        print(f"Error: {response.status_code} - {data.get('error',
'No error message')}")
        return None

api_key = "pk.60111b953ba9125fe871c9fd9d53c99f"
search_string = input("Enter the location: ")

result = get_geolocation(api_key, search_string)

if result:
    print("\nOutput:")
    for key, value in result.items():
        print(f"{key}: {value}")
```

Output:

```
>>>
========== RESTART: C:/Users/student/hhh
Enter the location:lucknow

output:
place_id: 242232459
lat: 26.8381
lon: 80.9346001
display_name: Lucknow, Uttar Pradesh, 226027, India
>>>
```

PRACTICAL 5

Aim: Develop application to download image/video from server or upload image/video

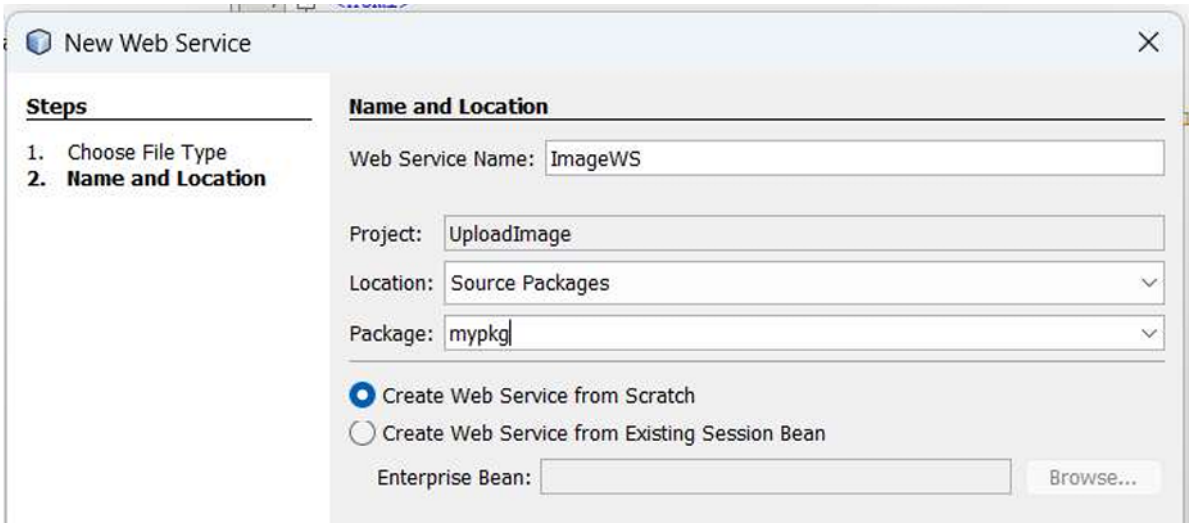
PART A: CREATE WEB SERVICE (SERVER SIDE)

Step 1: Create New project -> Java Web -> Web Application

Project name: UploadImage

Right click on UploadImage -> New -> Web Services -> RESTful Web Services

Web Service name: Image WS, Package name: mypkg



Step 2: Write the following code in web service class to accept image/video file,

```
package mypkg;
import java.io.*;
import javax.jws.Oneway;
import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;
import javax.xml.ws.soap.MTOM;

@MTOM(enabled = true, threshold = 60000)
@WebService(serviceName = "ImageWS")
public class ImageWS {

    @WebMethod(operationName = "upload")
    @Oneway
    public void upload(
            @WebParam(name = "Filename") String Filename,
            @WebParam(name = "ImageBytes") byte[] ImageBytes) {
```

```
        String filePath = "C:/MSC/upload/" + Filename;

        try {
            FileOutputStream fos = new FileOutputStream(filePath);
            BufferedOutputStream bos = new
BufferedOutputStream(fos);
            bos.write(ImageBytes);
            bos.close();
            System.out.println("Received file: " + filePath);
        } catch (Exception ex) {
            System.out.println(ex);
        }
    }

    @WebMethod(operationName = "download")
    public byte[] download(
            @WebParam(name = "Filename") String Filename) {

        String filePath = "C:/MSC/upload/" + Filename;
        System.out.println("Sending file: " + filePath);

        try {
            File file = new File(filePath);
            FileInputStream fis = new FileInputStream(file);
            BufferedInputStream bis = new
BufferedInputStream(fis);

            byte[] fileBytes = new byte[(int) file.length()];
            bis.read(fileBytes);
            bis.close();

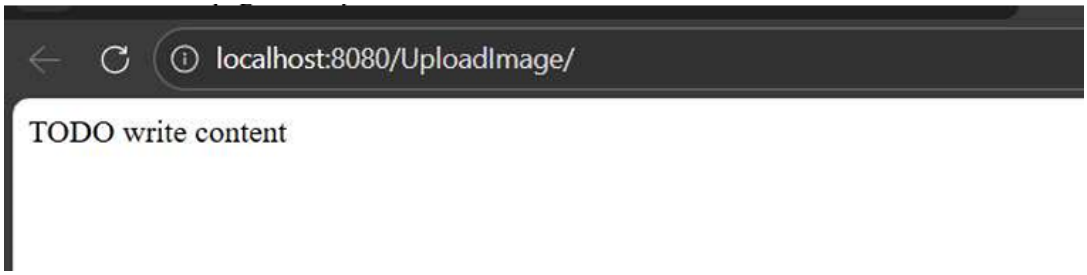
            return fileBytes;
        } catch (Exception ex) {
            System.out.println(ex);
        }

        return null;
    }
}
```

Right click on project -> Run and then Deploy

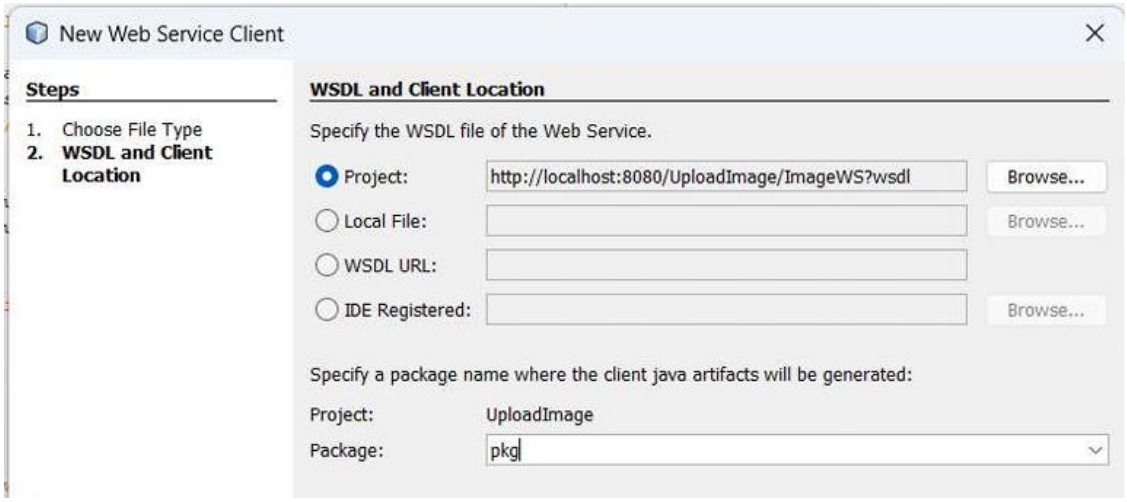
Step 3: Right click on RESTful Service -> Test RESTful Web Service

Output:



Step 4: Create Web server client -> Browse ImageWS from project

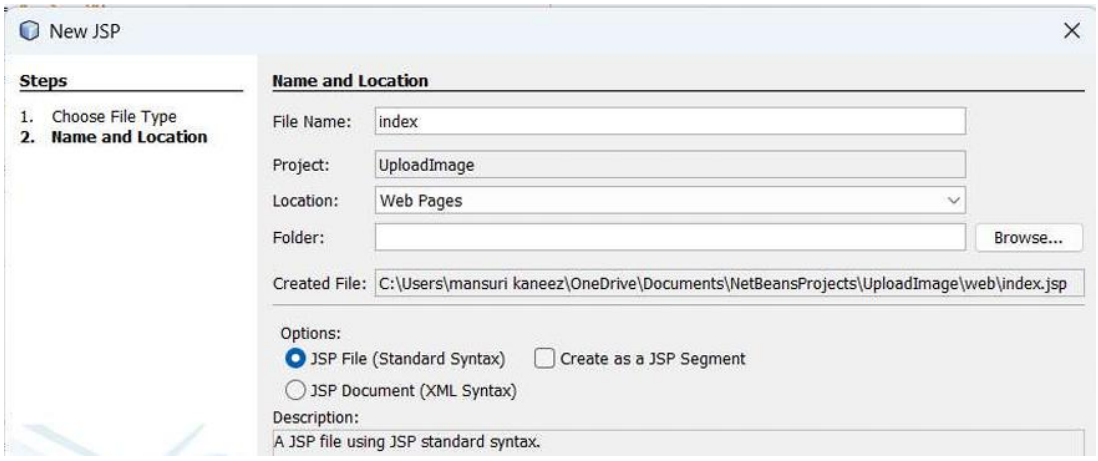
Package name: pkg



PART B: CREATE CLIENT (JSP PAGE)

Step 1: Right click on project -> New -> JSP

File name: index.jsp



Step 2: Write the following code in the file,

```
<%@page import="java.io.BufferedOutputStream"%>
<%@page import="java.io.FileOutputStream"%>
<%@page import="java.io.FileInputStream"%>
<%@page import="java.io.BufferedInputStream"%>
<%@page import="java.io.File"%>
<%@page import="javax.xml.ws.soap.MTOMFeature"%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
```

```
<!DOCTYPE html>
<html>
<head>
    <meta http-equiv="Content-Type" content="text/html;
charset=UTF-8">
    <title>Image Upload & Download</title>
</head>
<body>

    <h2>Upload Image to Server</h2>
    <hr/>

    <%-- Upload Section --%>
    <%
        try {
            pkg.ImageWS_Service service = new
pkg.ImageWS_Service();
            pkg.ImageWS port = service.getImageWSPort(new
MTOMFeature(60000));

            String filePath = "C:/Picture/flower.jpg";
            File file = new File(filePath);

            FileInputStream fis = new FileInputStream(file);
            BufferedInputStream bis = new
BufferedInputStream(fis);

            String filename = file.getName();
            byte[] imageBytes = new byte[(int) file.length()];
            bis.read(imageBytes);

            port.upload(filename, imageBytes);
            bis.close();

            out.println("<b>File Uploaded Successfully:</b> " +
filePath);

        } catch (Exception ex) {
            ex.printStackTrace();
        }
    %>

    <hr/>
    <h2>Download Image from Server</h2>
    <hr/>

    <%-- Download Section --%>
    <%
        try {
            pkg.ImageWS_Service service = new
pkg.ImageWS_Service();
            pkg.ImageWS port = service.getImageWSPort();

            String filename = "flower.jpg";
```

```
            String filePath = "C:/Picture/download/" + filename;

            byte[] fileBytes = port.download(filename);

            FileOutputStream fos = new FileOutputStream(filePath);
            BufferedOutputStream bos = new
BufferedOutputStream(fos);

            bos.write(fileBytes);
            bos.close();

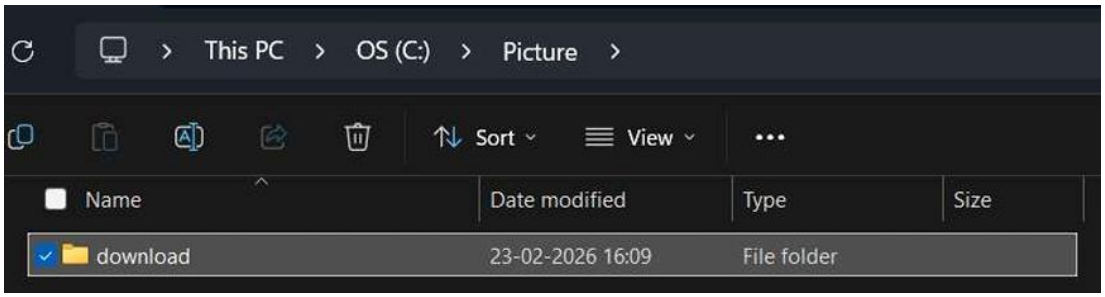
            out.println("<b>File Downloaded Successfully:</b> " +
filePath);

        } catch (Exception ex) {
            ex.printStackTrace();
        }
    %>
    <hr/>
</body>
</html>
```

PART C: CREATE FOLDER IN SYSTEM

Step 1: Go to This PC -> C Drive -> Create New folder named Picture

Inside Picture, create another folder named download



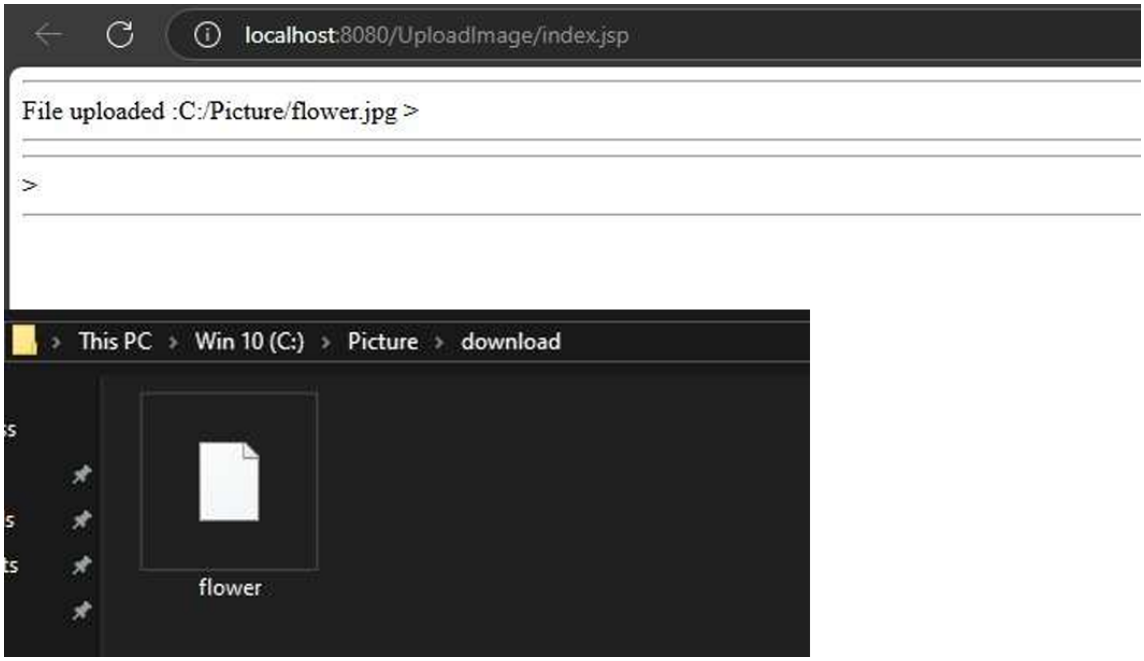
Step 2: Download any image of flower from Browser -> Save and rename it as flower.jpg inside C:\Picture\download

In Web Service Code, change the String file path to C Drive

Step 3: Deploy the project



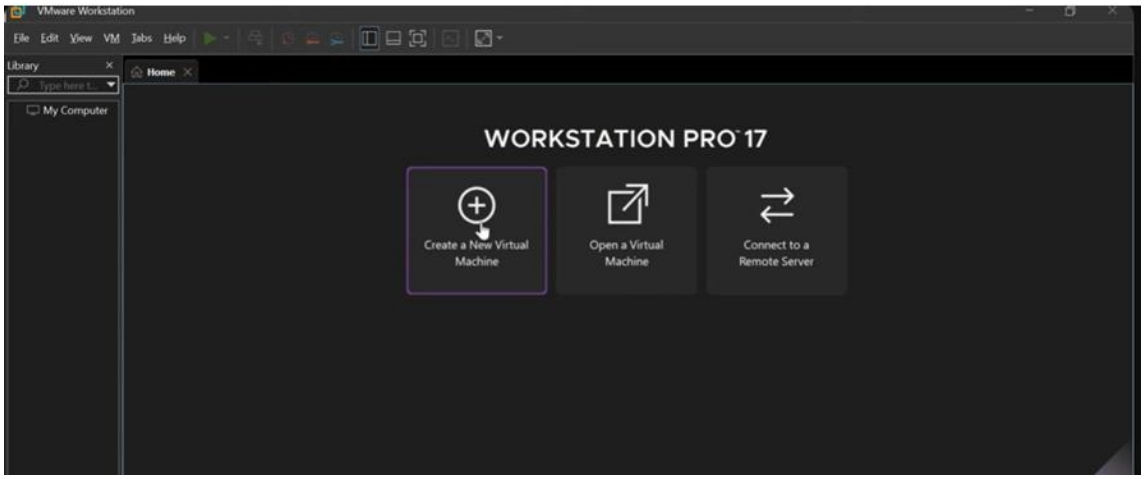
Step 4: Run index.jsp. Verify that file is stored in C:\Picture\download



PRACTICAL 6

Aim: Installation and Configuration of virtualization using KVM

Step 1: Open Vmware Workstation and start the steps



Step 2: Click on new Virtual Machine



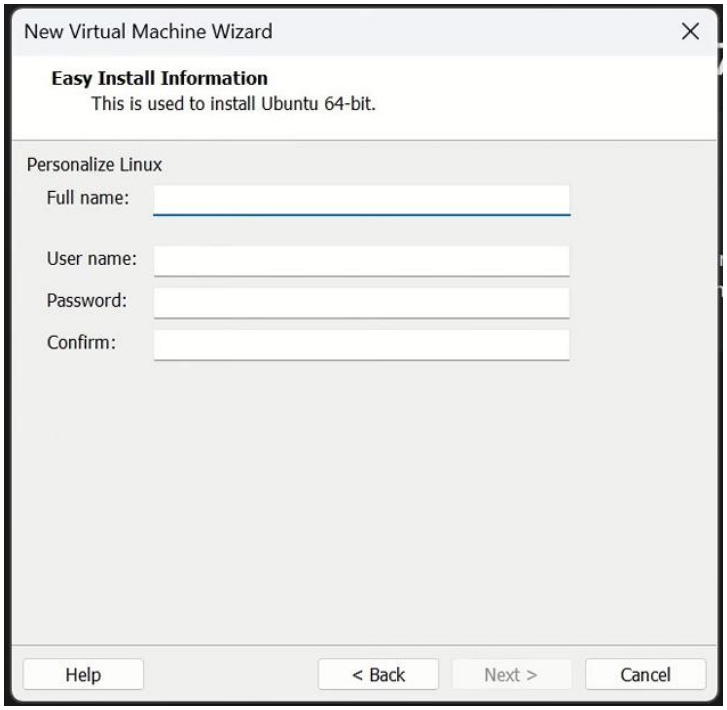
Step 3: If you don't have VMware and ubuntu download it from the following link

https://vmware-player.en.uptodown.com/windows#google_vignette

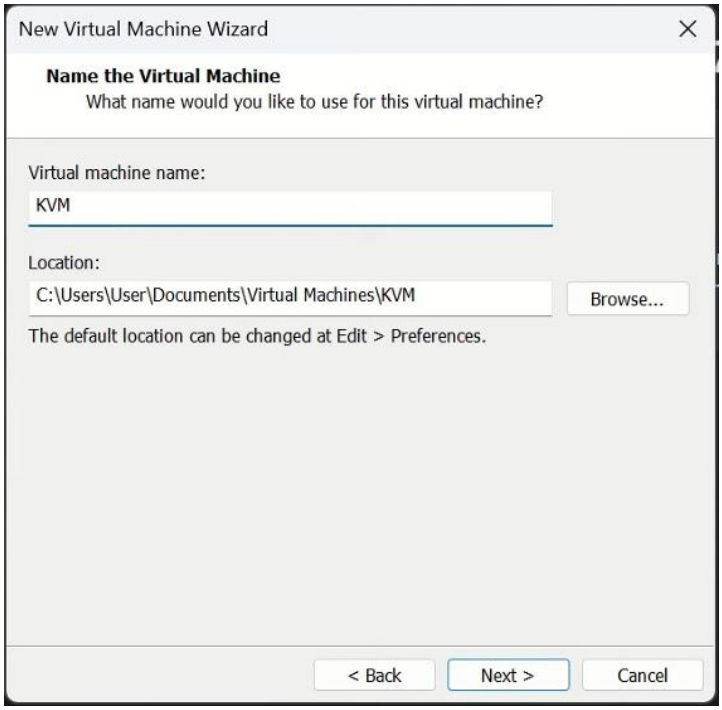
https://ubuntu.com/download/desktop?form=MG0AV3

Step 4: Once downloaded and installed Browse and select the ubuntu folder, expand the ubuntu folder until you come up to the setup

Step 5: Once the folder has been selected click on next, fill up the necessary details



Step 6: Click on next and give a name you your virtual machine



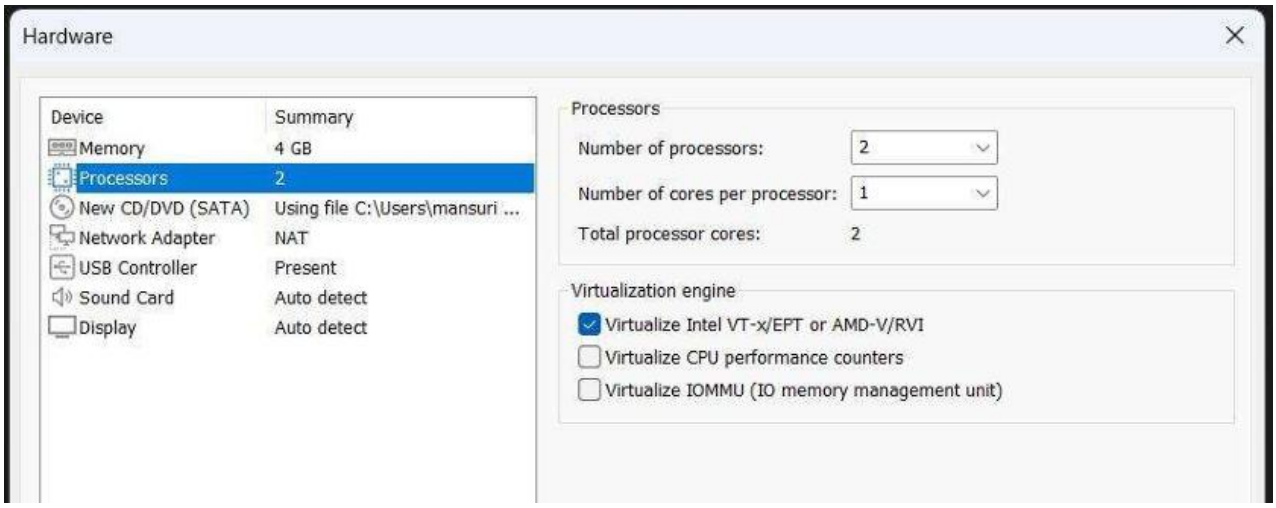
Step 7: Click on next and give disk size as 16 gb keep the spilt virtual disk into multiple file option selected and click on next

Then,



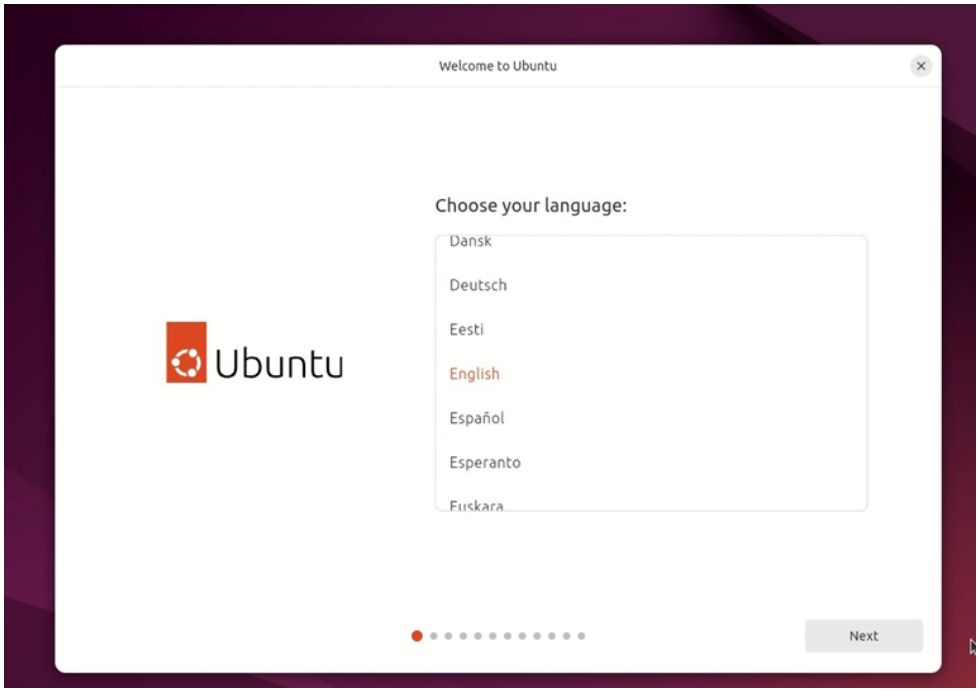
Step 8: Click on Customize Hardware

To enable KVM, select processor then tick the virtualize Intel VT



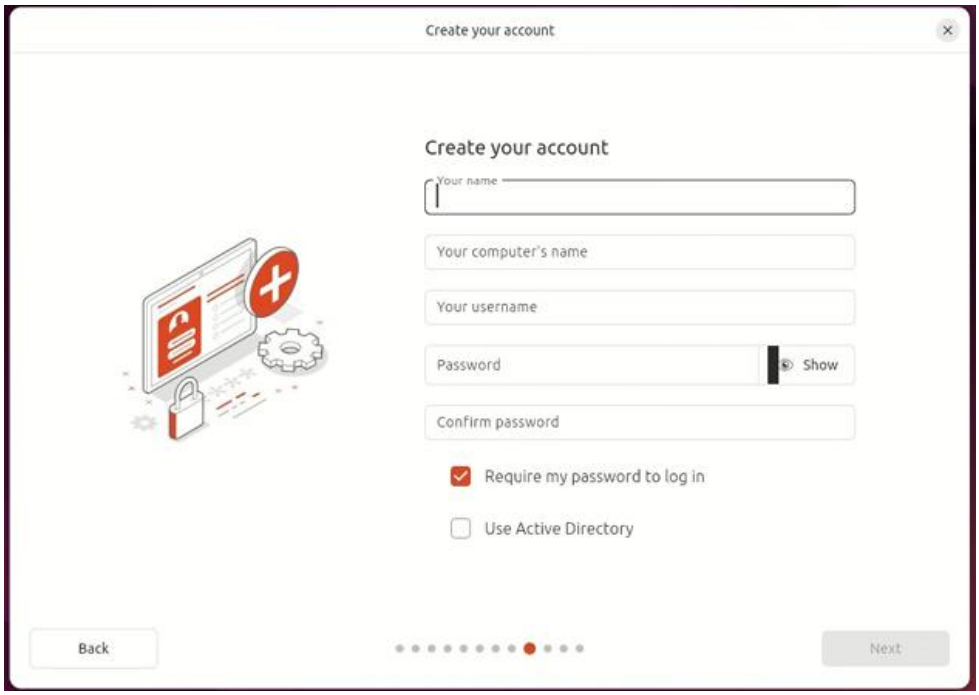
Step 9: Click on Finish to start your Ubuntu

NOTE: Your system should have 500 GB – 1TB space and 8 – 16 GB RAM for it to work



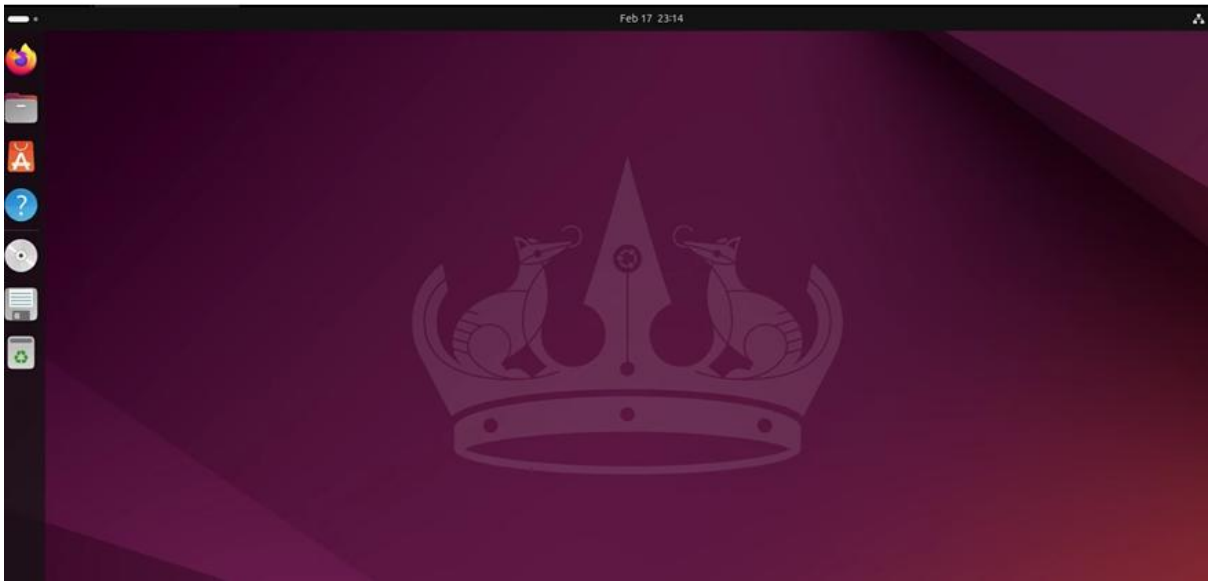
Keep clicking on Next until it asks for details

Step 10: Fill out all the details:



Click Next. At last click Install and then Restart now

Step 11: Login to your Ubuntu



Step 12: Now right click on the screen and select Open terminal

Enter this command to update your Ubuntu:

```
$ sudo apt update
$ sudo apt upgrade -y
```

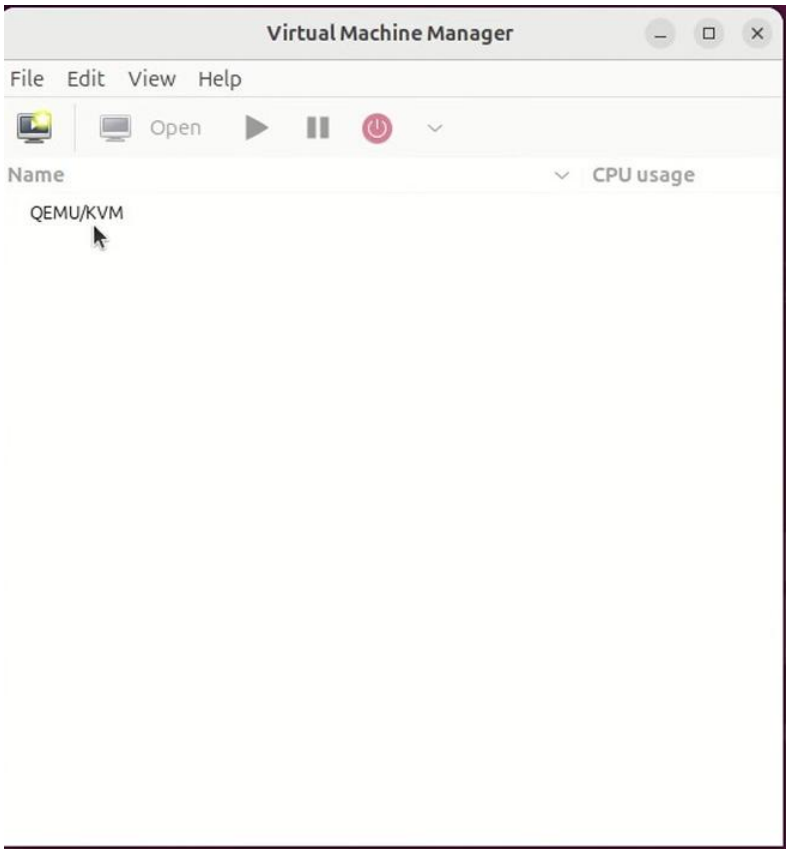
It will then ask for your password, Enter your password. After the upgrade, it will ask you to restart your ubuntu. Click on restart and login in your account once again and restart your terminal.

Now type the following commands one by one:

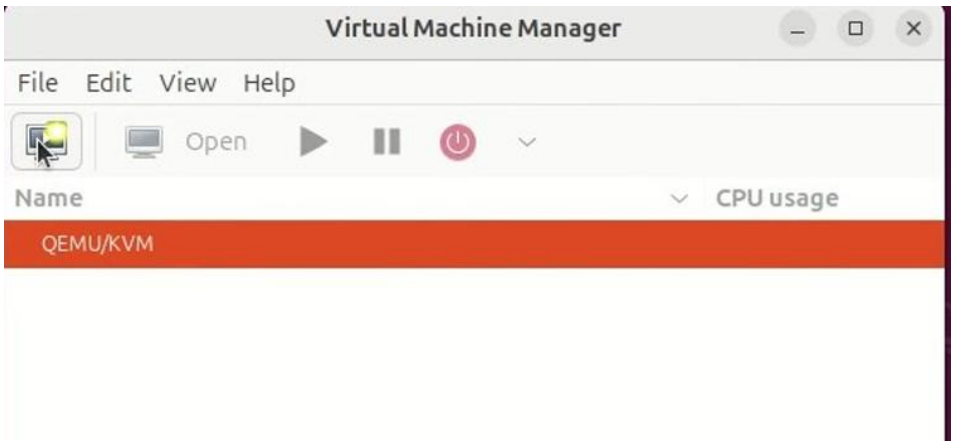
```
$ sudo apt install cpu-checker
$ sudo apt install qemu-kvm virt-manager libvirt-daemon-system libvirt-
clients bridge-utils -y
$ sudo systemctl enable libvirtd
$ sudo systemctl start libvirtd
$ sudo usermod -aG kvm noor (your user name)
$ sudo usermod -aG libvirt noor (your user name)
$ sudo systemctl status libvirtd
```



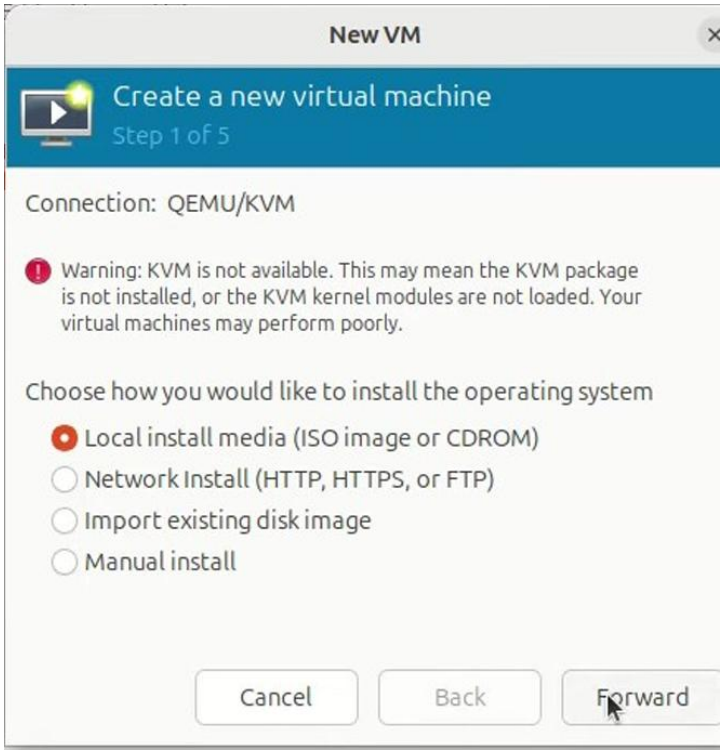
Step 13: Now go to show Apps and type vmm. If it's not getting connected, then Restart the system



Click on the icon as shown,

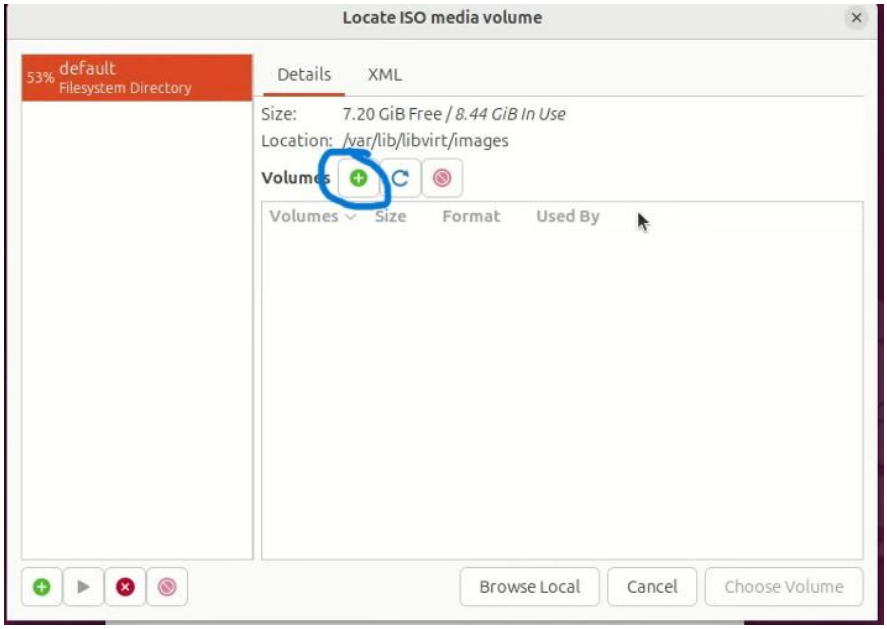


Now Click on Forward,

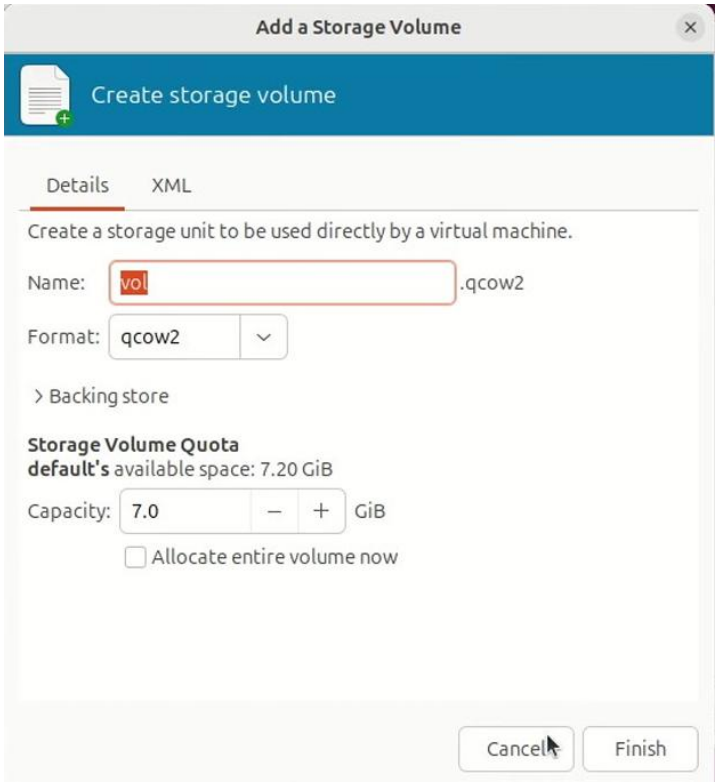


Click on Browse then click on the + sign,

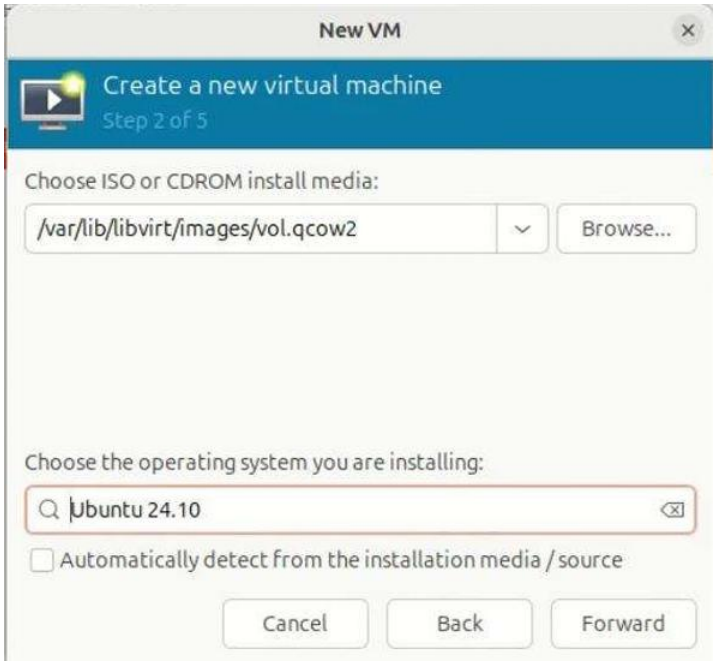
Click on Finish



Click on Choose Volume,

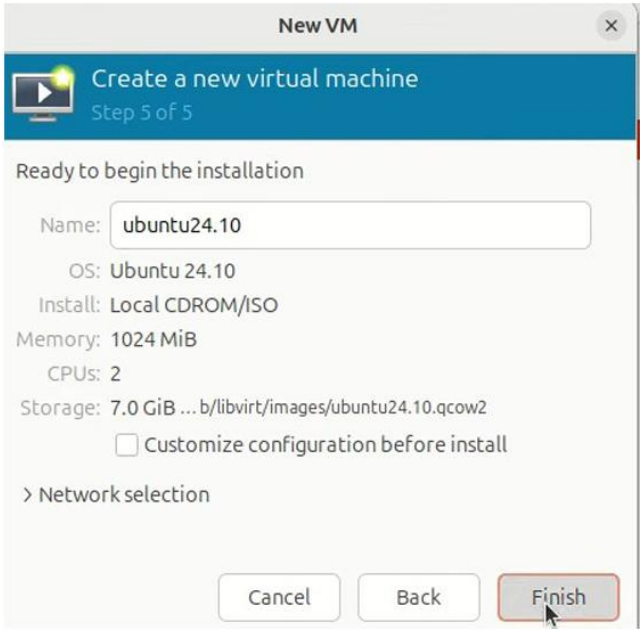


Now uncheck the box and search for your installed ubuntu and click forward,



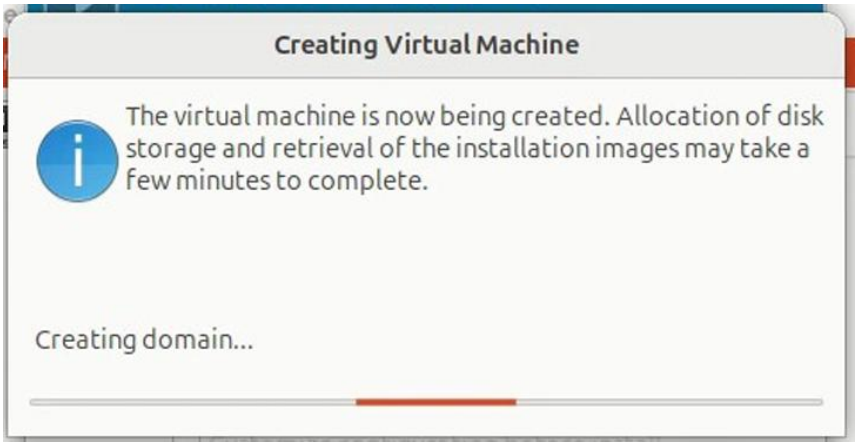
Now change your memory to 1024 and click forward

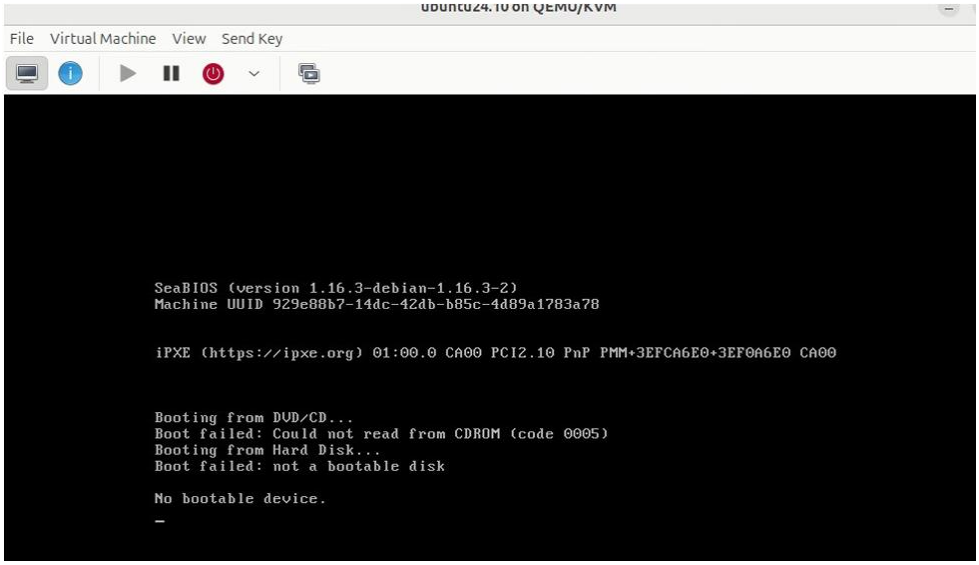
Change your disk memory to 7.0 GB then click forward



Click Finish.

Now your KVM is created,



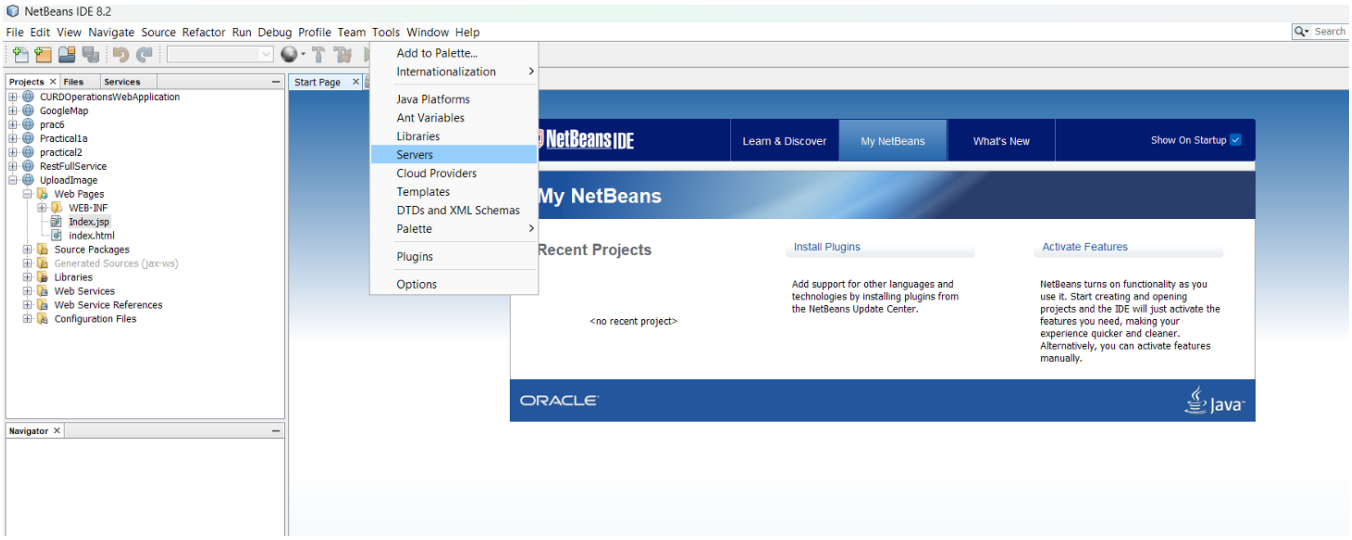


PRACTICAL 7

Aim: Develop and deploy a Servlet-based web application on Apache Tomcat.

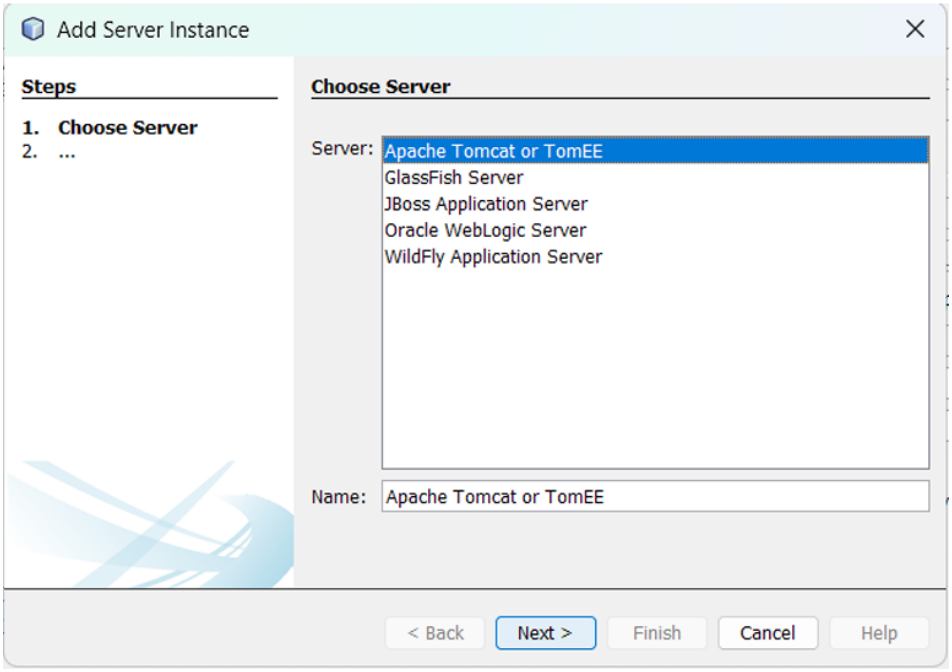
Step 1: Install Apache Tomcat

Download Tomcat 10 and extract

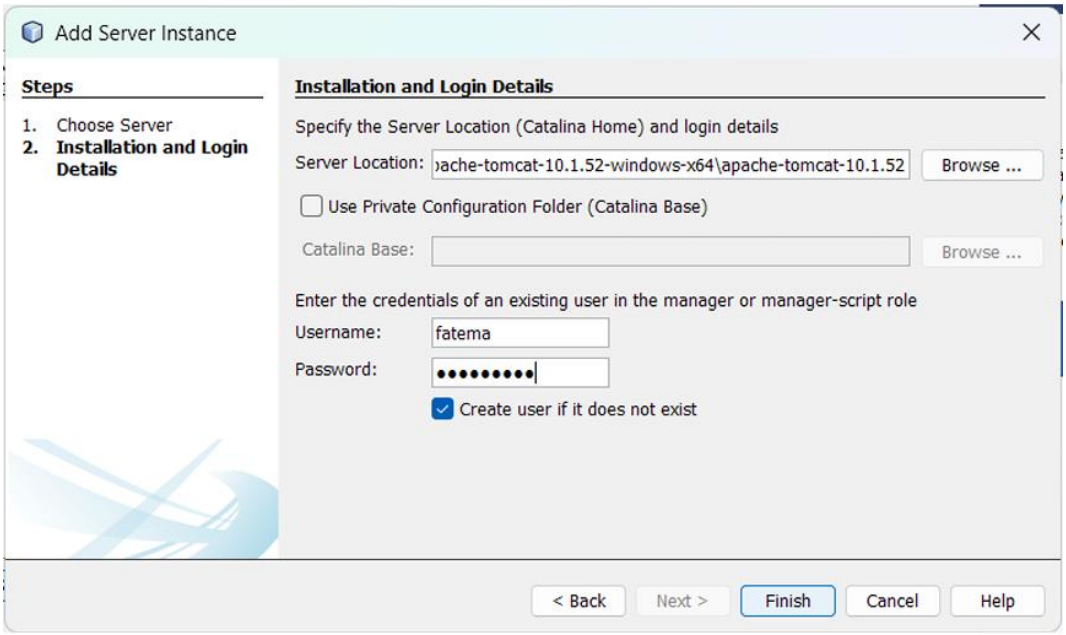


Step 2: Configure Tomcat in NetBeans

Go to Tools -> Servers -> Add Server -> Choose Apache Tomcat -> Select Tomcat folder



Browse the folder where your Apache Tomcat is. Give username and password.

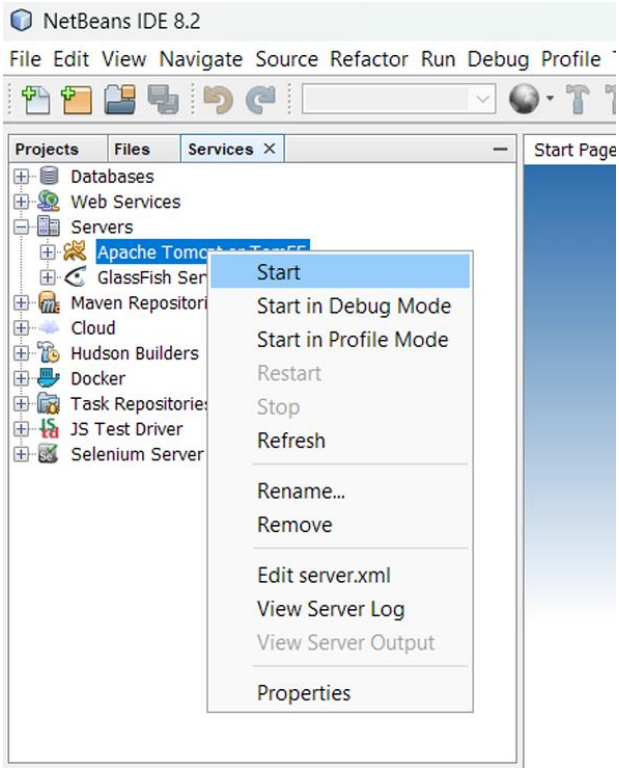


Step 3: Now Go to Services tab in NetBeans

• Expand Servers

• You should see: Apache tomcat

• Right click -> Start



Enter your username and password

Step 4: Create New project -> Select: Java with Ant → Web Application -> Click Next

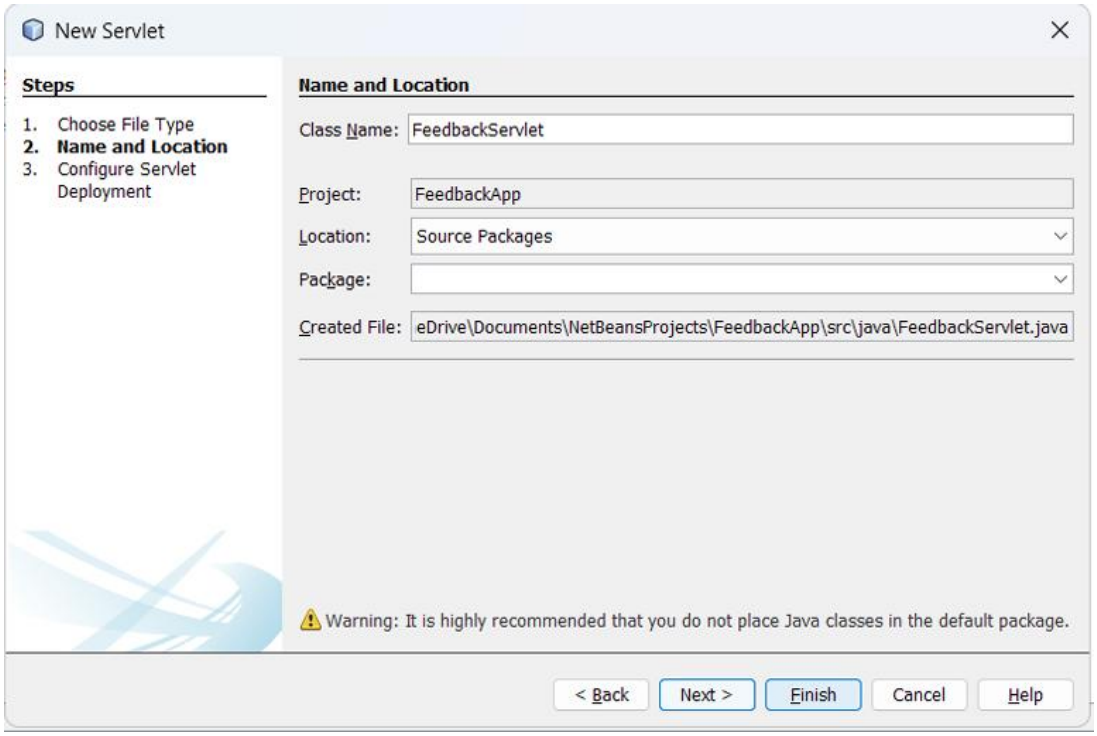
Project Name: FeedbackApp

Select Server: Apache Tomcat -> Finish

Create Servlet:

Right Click -> New Servlet

Servlet Name: FeedbackServlet



Step 5: Write the following code inside public class,

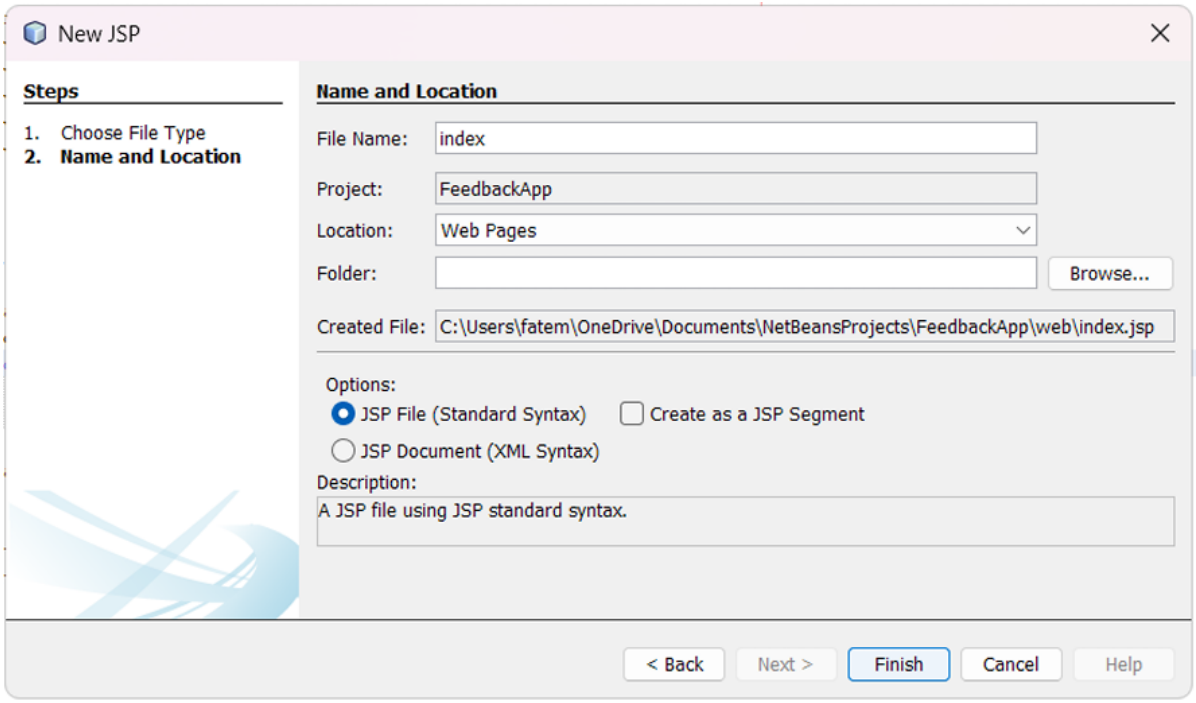
```
@WebServlet(urlPatterns = {"/FeedbackServlet"})
public class FeedbackServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
                          throws ServletException, IOException {

        String name = request.getParameter("name");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h2>Thank you " + name + "</h2>");
    }
}
```

Step 6: Create New JSP named index.jsp

Right click on Web pages folder -> New -> JSP. Name as index.jsp

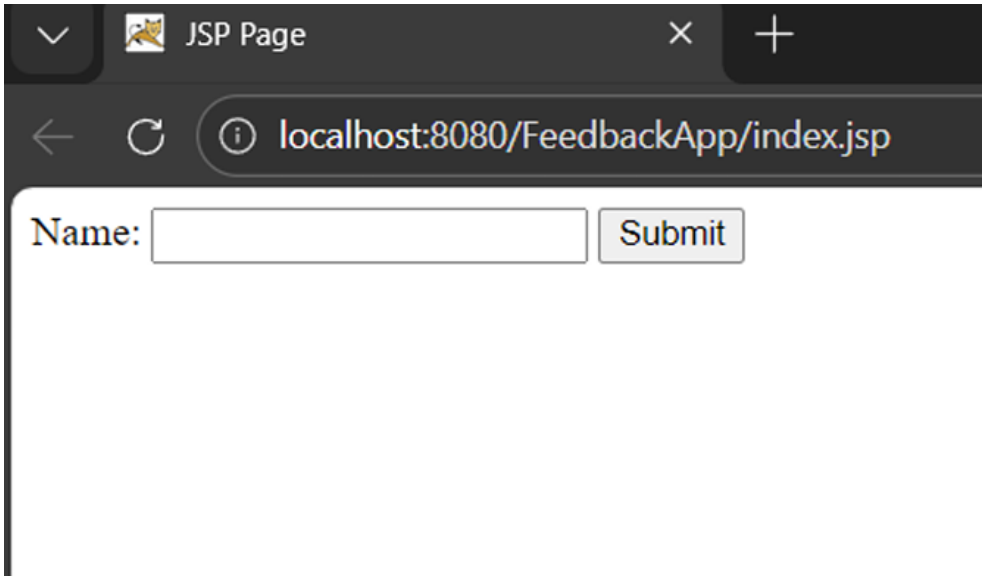


Write the following code inside body tag of index.jsp,

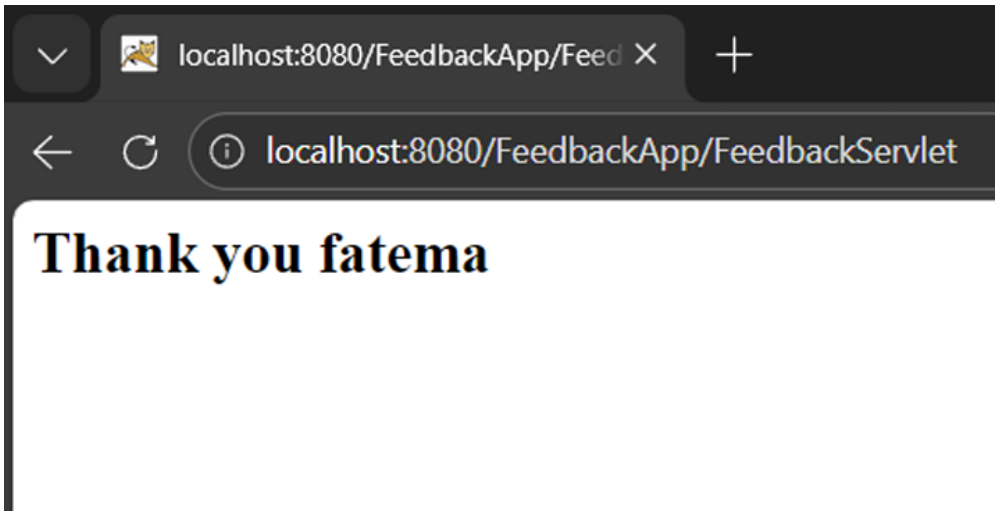
```
<body>
    <form action="FeedbackServlet" method="post">
        Name: <input type="text" name="name">
        <input type="submit">
    </form>
</body>
```

Step 7: Right click on project -> Deploy

Right click on index.jsp -> Run file



Enter your name and submit. You will get output as:

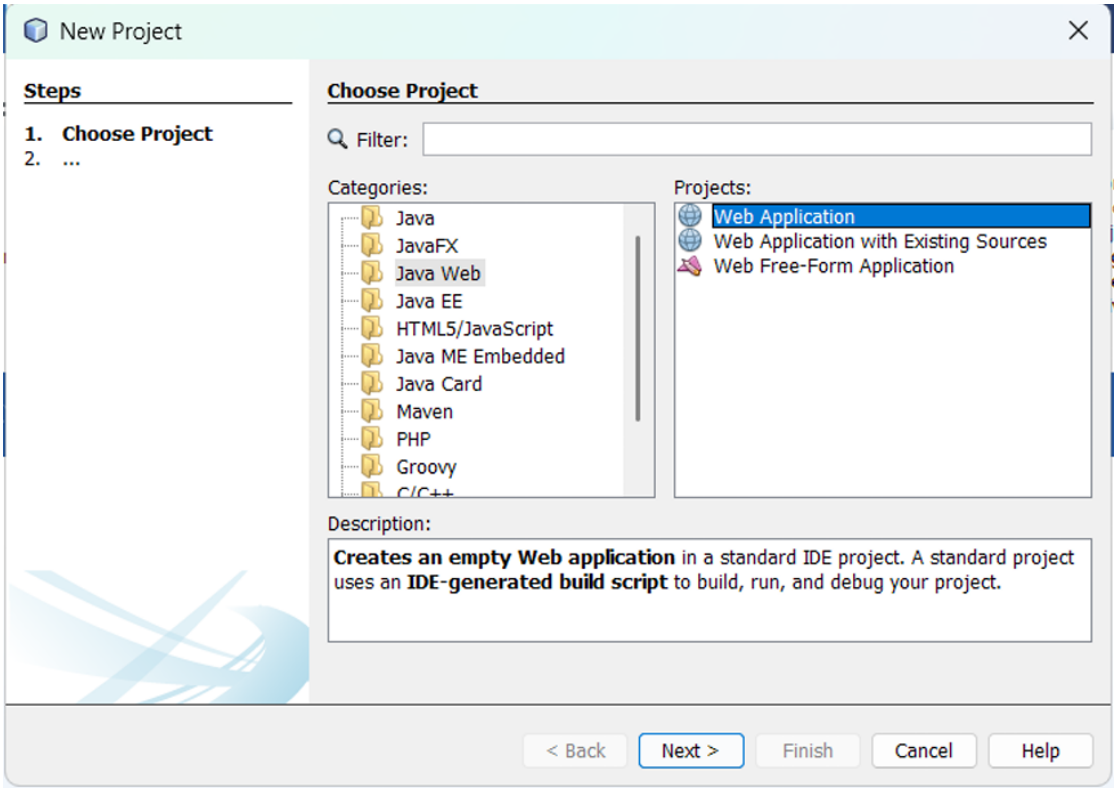


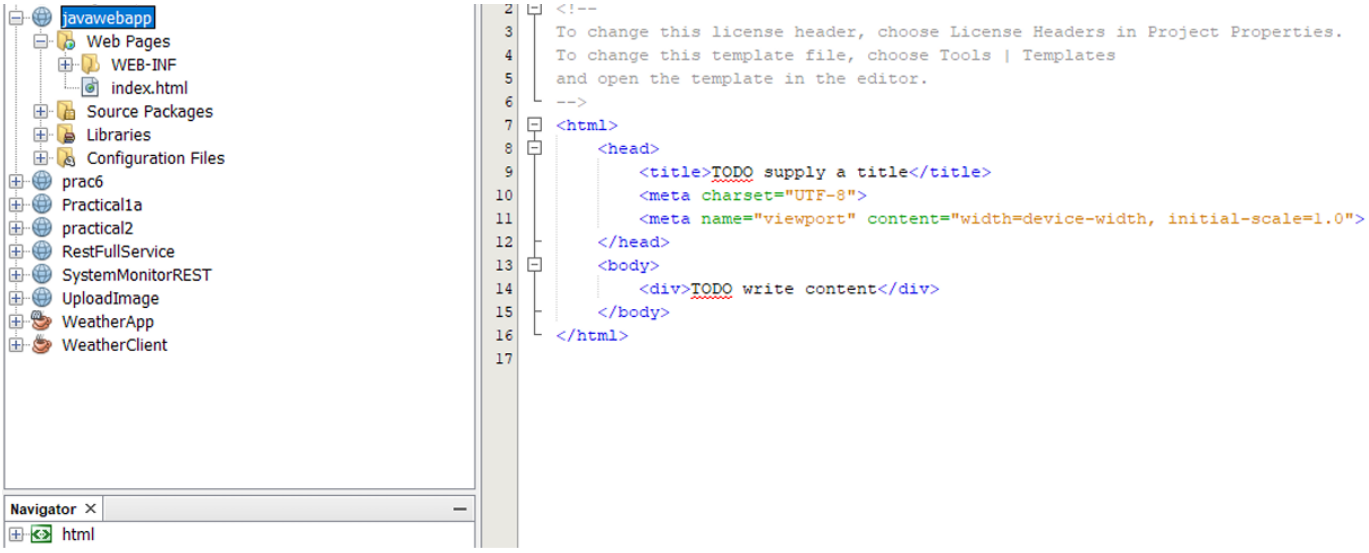
PRACTICAL 8

Aim: To develop a RESTful web service using Java that monitors and provides system resource usage, specifically memory and CPU usage, and returns this information in JSON format via HTTP GET requests.

Step 1: Create New project -> Java Web -> Web Application

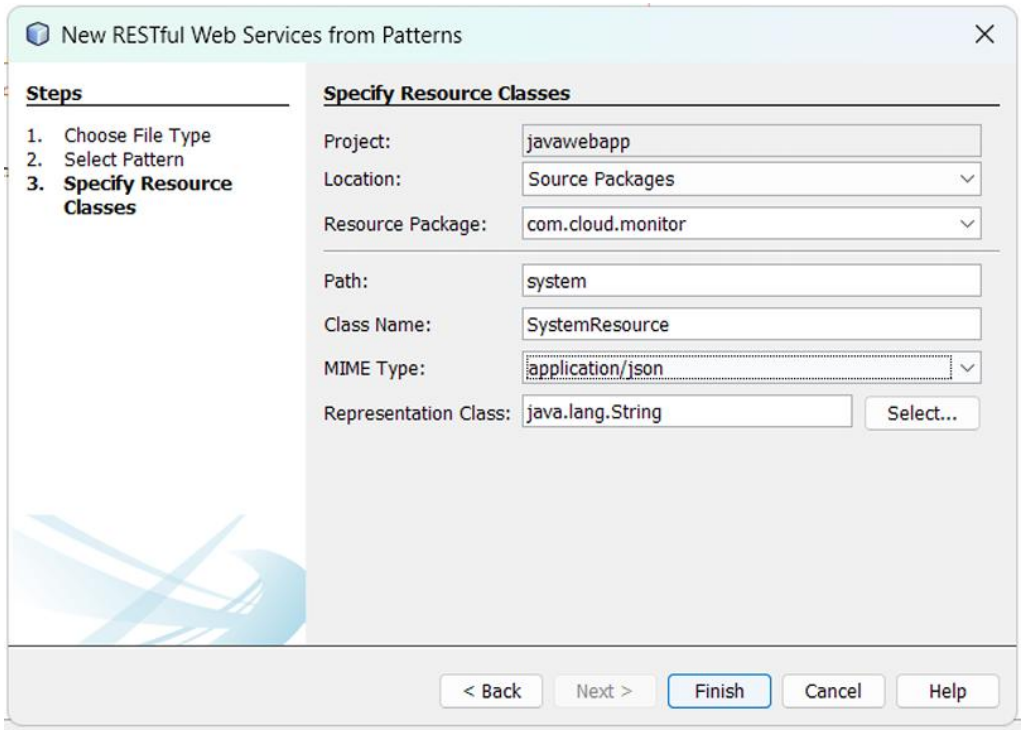
Project name: javawebapp





Step 2: Right click on Web INF -> New -> Select RESTful Web Services from patterns

Fill up the following details,



Step 3: Now in SystemResource.java file,

Write the following code:

```
package com.cloud.monitor;

import java.lang.management.ManagementFactory;
import javax.ws.rs.GET;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.core.MediaType;

@Path("system")
public class SystemResource {

    public SystemResource() {
    }

    // ----------------- MEMORY API -----------------
    @GET
    @Path("memory")
    @Produces(MediaType.APPLICATION_JSON)
    public String getMemory() {

        Runtime runtime = Runtime.getRuntime();

        long totalMemory = runtime.totalMemory() / (1024 * 1024);
        long freeMemory = runtime.freeMemory() / (1024 * 1024);
```

```
        long usedMemory = totalMemory - freeMemory;

        return "{"
                + "\"TotalMemoryMB\":\"" + totalMemory + "\","
                + "\"FreeMemoryMB\":\"" + freeMemory + "\","
                + "\"UsedMemoryMB\":\"" + usedMemory + "\""
                + "}";
    }

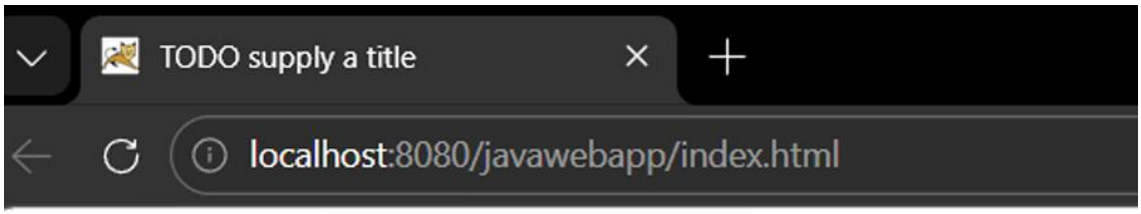
    // ----------------- CPU API -----------------
    @GET
    @Path("cpu")
    @Produces(MediaType.APPLICATION_JSON)
    public String getCPU() {

        com.sun.management.OperatingSystemMXBean osBean =
                (com.sun.management.OperatingSystemMXBean)
                ManagementFactory.getOperatingSystemMXBean();

        double cpuLoad = osBean.getSystemCpuLoad() * 100;

        return "{"
                + "\"CPULoadPercentage\":\""
                + String.format("%.2f", cpuLoad)
                + "\""
                + "}";
    }
}
```

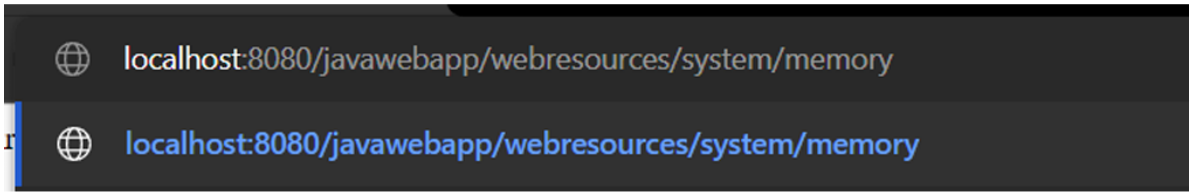
Step 4: Deploy the project and then run index.html file

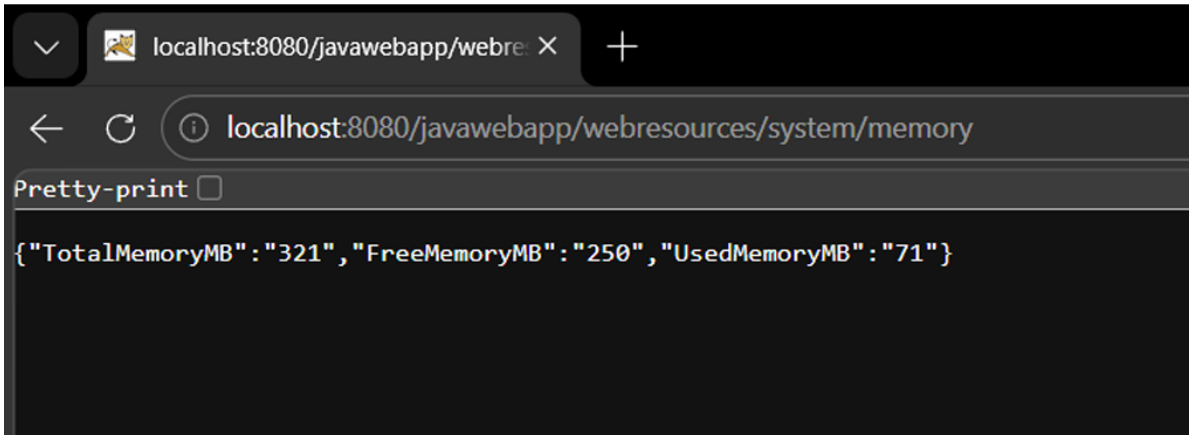


TODO write content

Step 5: After deployment, open browser and test:

- **Memory API:**





- **CPU API:**

